Budapest University of Technology and Economics
Faculty of Electrical Engineering and Informatics

Programming 3
(BMEVIIIAB00)

Java Text Editor Documentation

Final Project

Author
Diego Davidovich Gomes
NEPTUN Code: GHXLNI

December 4, 2025

Contents

1 Introduction **1**

2 Overview **1**
2.1 General Design . . . 1
2.2 Main Features . . . 2
2.3 Use Cases . . . 2
2.4 External Libraries and Code . . . 3

3 Software Architecture **3**
3.1 Structure . . . 3
3.2 Package Overview . . . 4
3.3 Model . . . 6
3.4 View . . . 6
3.5 Controller . . . 7
3.5.1 Main Controller . . . 8
3.5.2 Subcontrollers . . . 8
3.5.3 Service Layer . . . 9
3.5.4 Tool Components . . . 9

4 Sequence Diagrams **10**
4.1 View Word & Character Count . . . 10
4.2 Format Text . . . 10
4.3 Find and Highlight Text . . . 11
4.4 Edit Text . . . 12
4.5 Create/Open/Save Document . . . 12

5 Unit Tests **13**
5.1 Testing Strategy . . . 13
5.2 DocumentManagerTest . . . 13
5.3 FindToolTest . . . 13
5.4 FormattingActionTest . . . 14
5.5 LoadSaveServiceTest . . . 14
5.6 StatusBarToolTest . . . 15

6 References **16**

1 Introduction

Text editors are among the most fundamental and widely used software applications, providing users with essential tools for creating and modifying text. From simple note-taking apps to advanced word processors, it is without doubt that text editors play an important role in everyday life.

This project focuses on designing and implementing a simplified yet fully functional **rich-text editor** that supports common word-processing features. The application allows users to create, edit, format, and control text documents through a simple graphical interface. In addition to the basic editing and formatting capabilities, the program includes support for opening, saving, and creating documents, as well as searching using a Find tool.

The purpose of this project is to apply the concepts learned in the **Programming 3** course, such as java serialization, unit testing, Swing, object-oriented concepts, to build a complete application. The project is implemented in **Java 21** and built using **Maven**, which manages dependencies and the project structure. **Git** is used for version controlling and static analysis is performed by **SonarQube**.

2 Overview

This section provides a high-level overview of the application, summarizing its overall design, main components, and main ideas behind its implementation.

2.1 General Design

The application follows a simplified **Model-View-Controller** (MVC) architecture, organized by a **Document Manager**, which coordinates multiple open documents and enables editing in separate, independent windows. The **Model** represents the underlying document data, including the styled text, formatting attributes, and references the file which stores the document. The **View** exclusively provides the graphical user interface, composed of the editor pane, menu bar, toolbars, status bar, and various dialogs. It displays the document to the user and exposes all available operations through the components (**Note**: not every component is currently implemented, e.g. bullet and numbered lists). The **Controller** acts as the coordinator. It processes the users actions coming from the view, translates them into operations on the model, and the interface is updated. The controller manages the different subsystems of the application, such as actions, tools, and services, and it is responsible for attaching them to the correct component on the view.

- **Actions**: implements custom **Swing Actions** for user-initiated commands of formatting.
- **Tools**: contain the logic for features such as text searching, zooming, formatting, and status bar updates.
- **Services**: encapsulate specialized functionality such as saving and loading documents.

This structure improves maintainability by separating responsibilities into smaller units, while the document manager ties the whole architecture together.

2.2 Main Features

The application provides the main functionalities expected from a text editor, implementing all features previously written in the specification and several additional improvements introduced during development:

- **Rich Text Editing**: supports formatting through different fonts, font sizes, text color, highlighting, bold, italic, underline, strike-through, subscript, superscript, and paragraph alignment.
- **Document Management**: create new documents, opening existing ones, and saving them into **.rtf** files using Java serialization.
- **Find**: allowing users to search for text occurrences within the document.
- **Zooming and Scaling**: made possible through the custom editor kit that enables dynamic text scaling.
- **Status Bar**: displays word and character count, and contains the zoom slider.
- **Menu bar and Toolbar**: offers quick access to formatting tools, file operations and editing commands.
- **Multiple Document Windows**: managed by the document manager to allow independent document editing.

2.3 Use Cases

The use cases of the application correspond to those previously defined in the project specification. The supported operations include document creation, opening, saving, text editing, formatting, and searching. The original use case diagram from the specification is included here without repeating the detailed descriptions.

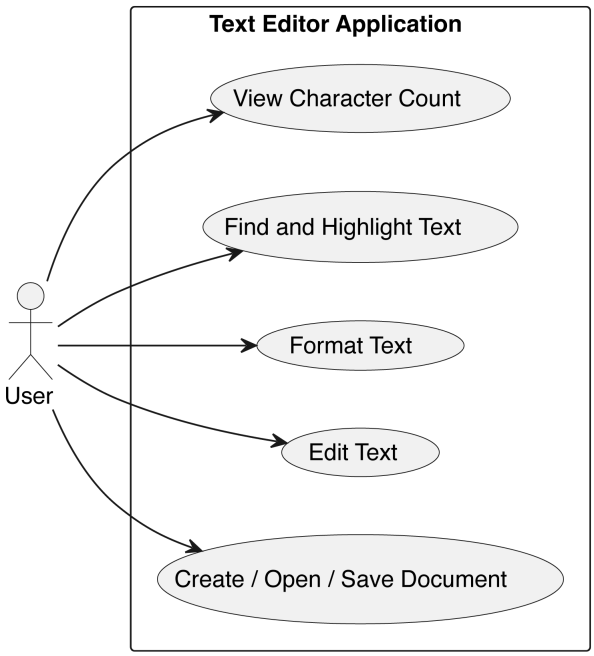

Figure 1: Use case diagram of the project

2.4 External Libraries and Code

The application uses the **FlatLaF** library to provide a modern and consistent appearance for the application. It is used for replacing the default **Swing Look and Feel** with a more contemporary flat design, and the application specifically uses the **FlatLaF macOS Dark** theme. On macOS, the application also benefits from FlatLaF's support for native integration features, such as moving the menu bar to the top of the screen. Additionally, **FlatLaF Extras** are used to enable the integration of SVG icons throughout the menus and toolbars.

In addition, the project incorporates a custom made editor kit (`ScaledEditorKit`, extending `StyledEditorKit`) originally written by **Stanislav Lapitsky**. The code enables dynamic text scalling and zooming, while solving issues related to inaccurate caret positioning by replacing part of Swing's internal text rendering classes with float-based implementations. Few modifications were made for integration, mainly the removal of **reflection**-based access that are no longer allowed in modern Java. Full credit is given to the original author [1].

3 Software Architecture

This section outlines the internal structure of the application, describing its main components and how they interact to provide the required functionality.

3.1 Structure

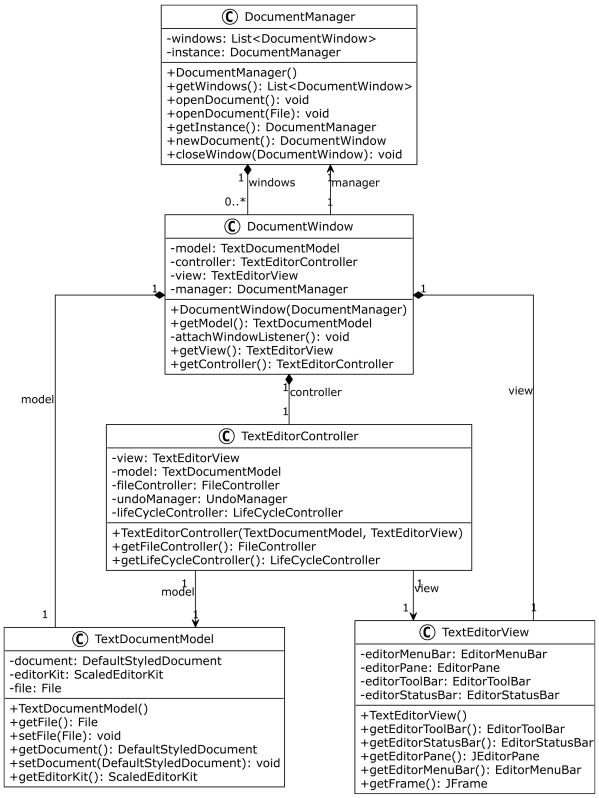

Figure 2: High-level UML class diagram of the text editor (Link)

The application follows a **Model-View-Controller** architecture at the level of each document window, while an additional management layer handles multiple documents. As shown in Figure 2, at the top of this structure is the `DocumentManager`, which is responsible for creating, keeping, and closing the windows. It maintains a list of `DocumentWindow` instances, effectively being the owner of all open documents. Each time the users creates a new document or opens an existing file, it constructs a new `DocumentWindow` and adds it to the list. The class itself is implemented as a **singleton**, meaning it provides a single globally accessible instance for the entire application and ensuring there is only one centralized manager.

The `DocumentWindow` wraps the MVC structure for a single document. As the name suggest, it is composed of a model (`TextDocumentModel`), a view (`TextEditorView`), and a controller (`TextEditorController`). The model holds the underlying document data, while the view is solely responsible for its graphical representation and providing the user with an interface to perform operations on the document. The controller ties the model and the view together by interpreting the user interactions coming from the view, translating them into operations on the model, and retrieves data from the model to update the interface. The principle of this structure is to keep the view as "dumb" as possible, purely visual and free of business logic, while the model remains strictly data-focused.

Note: although this separation is intended, **Swing** is not a pure MVC framework. The view and model naturally hold references to each other through Swing components such as `JEditorPane` and `Document`. Nevertheless, the design of the application preserves the MVC intent by ensuring that all logic passes through the controller.

3.2 Package Overview

To support a modular architecture and separate responsabilities into smaller units, the project is organized into multiple packages. Each package groups together related classes that work toward a common functionality, helping maintain clarity and improving maintainability. Figure 3 below presents the tree structure of the package organization.

```
com.texteditor
├── actions
├── config
├── controller
│   ├── actions
│   ├── management
│   ├── services
│   └── tools
├── editorkit
│   └── scaling
├── manager
├── model
└── view
    ├── components
    └── widgets
```

Figure 3: Package structure

1. **com.texteditor.actions**: defines enums that define the set of available actions in each category (e.g. formatting). They allow the application to refer to actions in a consistent way. These enums are used in the view to build an `EnumMap` that links each action to its component, making it easier to the controller to attach actions and avoiding the need for many individual getters.

2. **com.texteditor.config**: includes the class responsible for application configuration. These settings adjust global GUI behavior, apply platform-specific visuals.

3. **com.texteditor.controller**: contains the main controller logic of the application. The `TextEditorController` is the top level class and connects the smaller controller components found in the subpackages.

   - **actions**: groups all custom **Swing Actions** used for the text formatting. These actions are triggered through the user interface and are used by the `FormattingTool` in the tools package.
   - **management**: contains the `FileController`, which handles opening and saving documents, and the `LifeCycleController`, which tracks whether a document is saved, updates the window title, and manages document closing.
   - **services**: offers the `LoadSaveService` responsible for loading and saving the document using a `RTFEditorKit`.
   - **tools**: defines the controller-level operations for formatting, searching, status updates, and zooming. It implements the logic for these features and connects them to the GUI by attaching the necessary action listeners and actions.

4. **com.texteditor.editorkit**: includes the custom **ScaledEditorKit**, located in the `scaling` subpackage.

5. **com.texteditor.manager**: provides the higher-level managers that enable multple documents to exist simultaneously. `DocumentWindow` wraps the editor's MVC structure, while `DocumentManager` manages the creation and tracking of all open documents.

6. **com.texteditor.model**: holds the core data of the application. It includes the `TextDocumentModel`, which represents the underlying document data, and `TextDefaults`, which defines the default formatting values.

7. **com.texteditor.view**: encapsulates all graphical user interface components and provide access points for the controller. The main class, `TextEditorView`, assembles these user interface elements into a single coherent view of the application.

   - **components**: groups the classes that build the main parts of the graphical user interface, such as the menu bar, editor pane, status bar, and toolbar. These are implemented as extensions of Swing components, allowing for customization.
   - **widgets**: includes the classes that implement custom GUI widgets and components, such as color selection buttons, dialogs, and other composite interface elements.

3.3 Model

The model component is represented by the `TextDocumentModel` class, which serves as the data layer within the architecture. It is solely responsible for storing the document content, formatting attribute, and associated metadata. The implementation centers on two core Swing components: a `DefaultStyledDocument` which according to the documentation is "a document that can be marked up with character and paragraph styles in manner similar to the Rich Text Format", and a custom "ScaledEditorKit" that extends Swing's StyledEditorKit to enable the zoom functionality. Together, these components provide formatting capabilities and support for dynamic text scaling.

The model additionally maintains a `File` reference to track the document responsible for storing the data. Default formatting values are externalized to the `TextDefaults` utility class.

Following a minimalist design, the model contains no business logic or user interface dependencies beyond necessary Swing interfaces. Figure 4 illustrates the structure and relationships of the model component and its supporting classes.

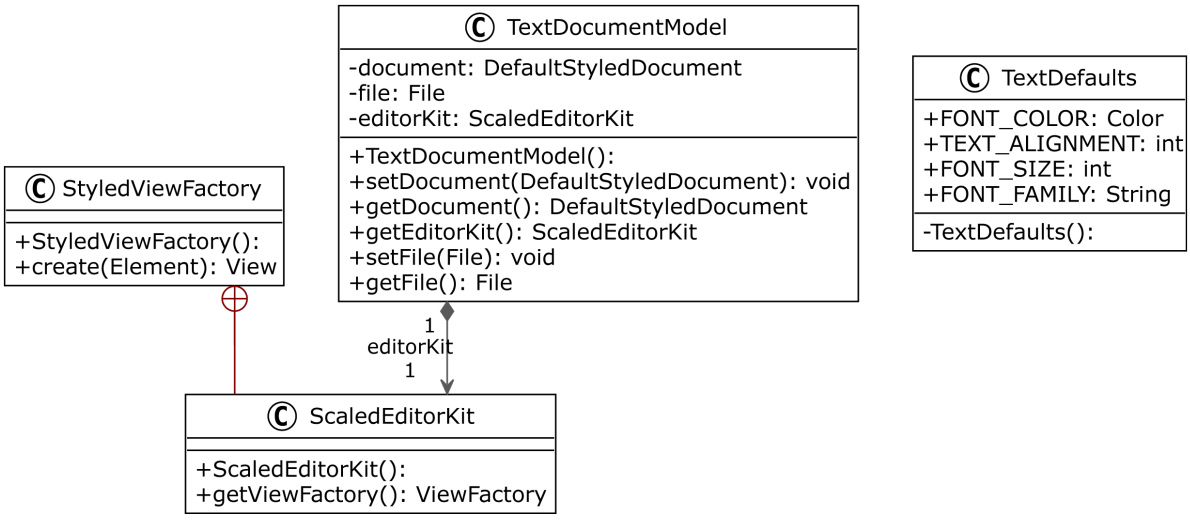

Figure 4: UML class diagram of the classes composing the Model (Link)

3.4 View

The view component serves as the presentation layer, responsible for rendering the graphical user interface and providing user interaction points. This separation ensures that all visual elements are separated from the application logic and data management.

The primary view class, `TextEditorView`, extends Swing's `JFrame` to create the main application window. It arranges the assemby of smaller, specialized GUI components, namely `EditorToolBar`, `EditorStatusBar`, `EditorPane`, and `EditorMenuBar`, and arranges them within the the appropriate layout. This decomposition into smaller modules manages the complexity of numerous interface elements while maintaining clarity and facilitating individual component access.

As described in the package overview (Section 3.2), the view is organized into `components` and `widgets` subpackages. The `components` implement major interface sections, while `widgets` provides custom GUI elements like `ColorSplitButton`, `ZoomSlider`, and `FontsComboBox`

Each major component provides structured access points through `EnumMap` collections, allowing for efficient access and avoiding multiple getters (e.g., for the toolbar, which would otherwise require more than 18 individual getter methods). This design enables the controller to attach action listeners and apply operations, promoting cleaner code organization. This approach ensures that the view remains focused on presentation concerns while providing the controller with access points for user interaction handling.

Figure 5 illustrates the structure and relationships of the view components and its supporting classes

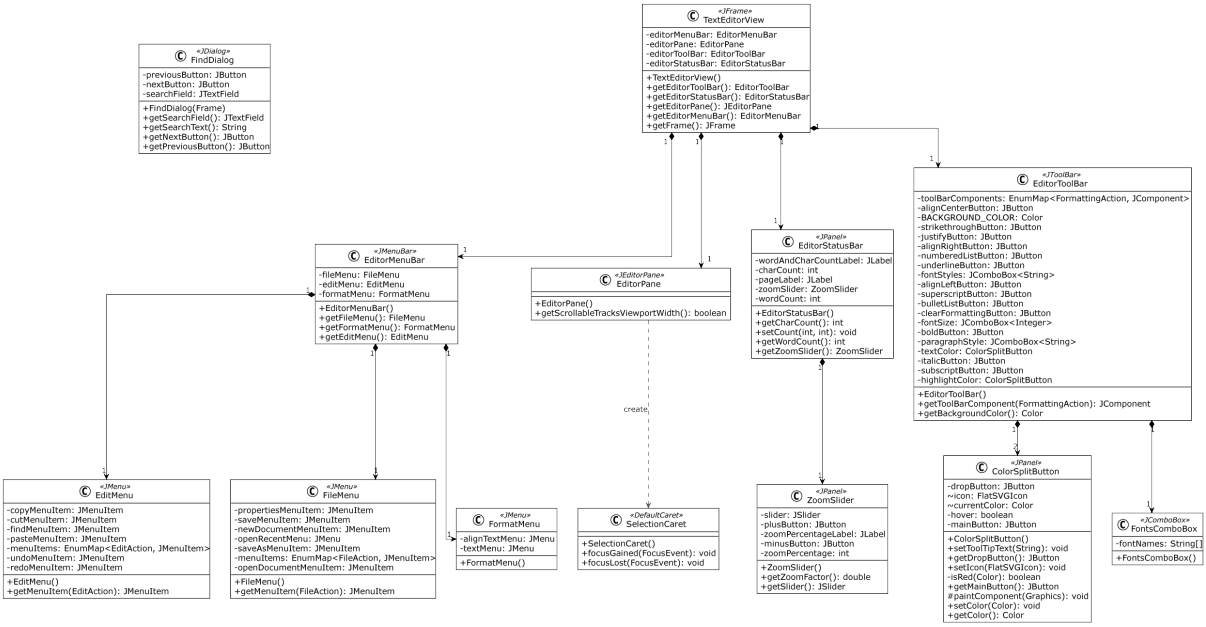

Figure 5: UML class diagram of the classes composing the View (Link)

Note: Some components are present in the codebase but not currently utilized in the application. For example, `TitleBarToolBar` was implemented but not integrated due to compatibility issues with macOS, and `ParagraphStyleComboBox` remains as a placeholder for future functionality expansion.

3.5 Controller

The controller serves as the central coordinating component within the system, functioning as the application's logical core. It intermediates all the interactions between the model and the view, translating user input into model operations and updating the view accordingly. This separation ensures that business logic remains decoupled from both data management and presentation concerns. Figure 6 illustrate the structure and relationships of the controller component and its supporting classes.

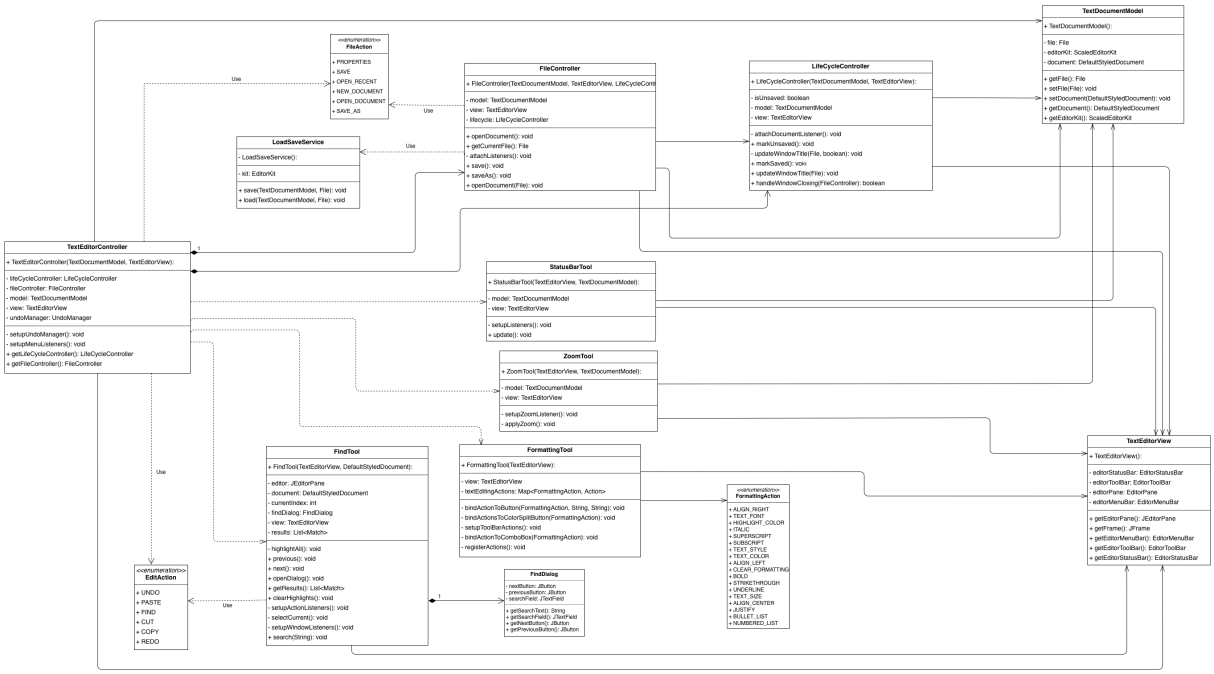

Figure 6: UML class diagram of the classes composing the Controller

3.5.1 Main Controller

The primary controller, `TextEditorController`, serves as the central coordinating point for each document window. It maintains references to both the model (`TextDocumentModel`) and view (`TextEditorView`), establishing the connections necessary for the MVC operation while preserving separation of concerns. Upon initialization, the controller performs several setup tasks:

- Establishes the connection between the view's editor pane and the model's document and editor kit
- Creates the specialized subcontrollers for distinct functional areas
- Initializes various tools that provide specific application features
- Setting up the undo/redo functionality using Swing's `UndoManager`
- Attach the different actions listeners to menu items and toolbar components

The controller delegates complex tasks to specialized components rather than concentrated in a single class.

3.5.2 Subcontrollers

To manage complexity and maintain separation of cocnerns, the controller includes specialized subcontrollers.

`FileController` manages document operations, including saving, loading, and file creation. It utilizes the `LoadSaveService` for RTF-based serialization, presenting file dialogs to users and handling file management. The RTF format imposes some limitations on supported formatting features, however it provides a standardized and "good enough" solution.

`LifeCycleController` monitors document state and manages window lifecycle operations. It attaches document listeners to detect unsaved changes, tracks the document's saved/unsaved status, updates window title to show the current state, and handles the window closing, asking the users to save unsaved changes.

3.5.3 Service Layer

The `LoadSaveService` provides static methods for document serialization and deserialization using Swing's `RTFEditorKit`. This service layer encapsulates the file input/output operations and RTF formatting.

3.5.4 Tool Components

The controller architecture includes specialized tools that implement specific application features.

`ZoomTool` manages text scaling by attaching change listeners to the zoom slider in the status bar. It retrieves zoom factors from the slider interface and applies them to the document through custom properties, triggering view updates.

`StatusBarTool` monitors document content through document listeners, automatically updating character and word count displayed in the status bar.

`FormattingTool` serves as the central coordinator for text formatting operations. It creates and manages custom Swing actions for various formatting features (e.g. bold, italic, color, alignment, etc.) and binds these actions to corresponding toolbar and menu components. The tool utilizes the view's `EnumMap` access points to efficiently connect formatting logic with interface elements.

Note: The relations between the `FormattingTool`, view components, and the action classes is omitted from the main class diagram (Figure 6) for simplication. The detailed structure is presented in Figure 7 below.

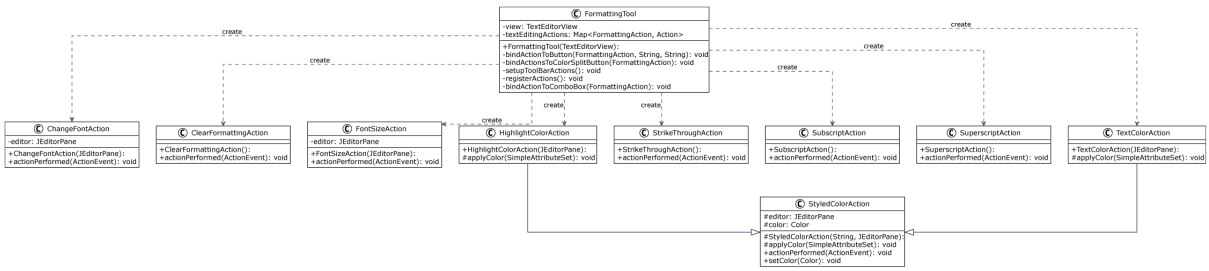

Figure 7: UML class diagram detailing FormattingTool, view, and action (Link)

`FindTool` implements text search functionality through a dedicated dialog interface. It provides methods for text searching, navigation between matches, and visual highlighting of results, with appropriate event listeners to the dialog controls.

4 Sequence Diagrams

These diagrams focus on the main flow of each operation without getting into excessive detail about internal method calls. They show the key interactions between the user interface, controllers, services, and model components, maintaining a level of abstraction appropriate for this documentation.

4.1 View Word & Character Count

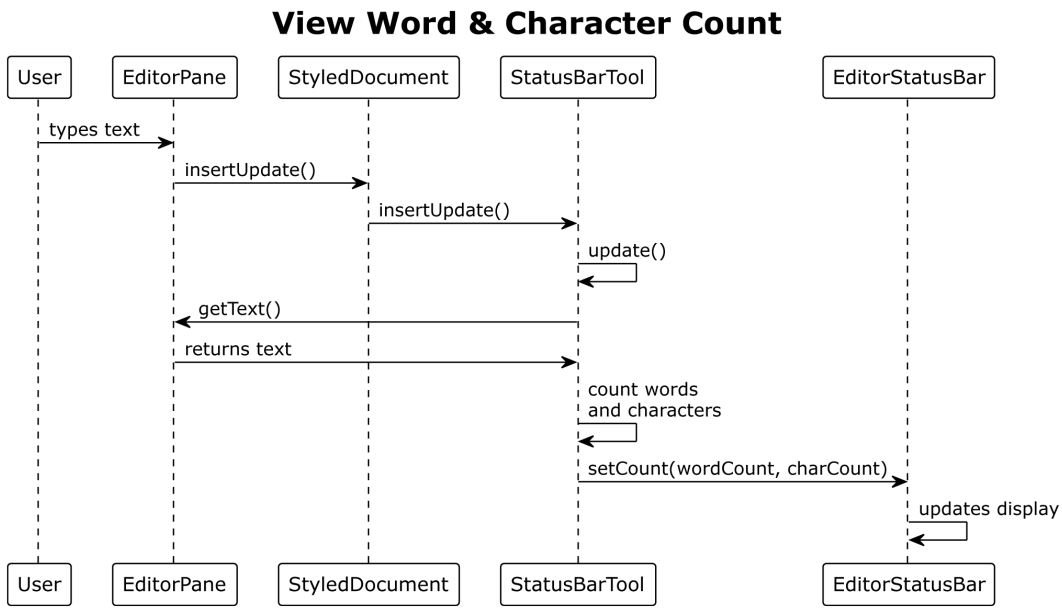

Figure 8: This diagram illustrates the real-time character and word count update process, showing how document statistics are tracked as users modify text content (Link)

4.2 Format Text

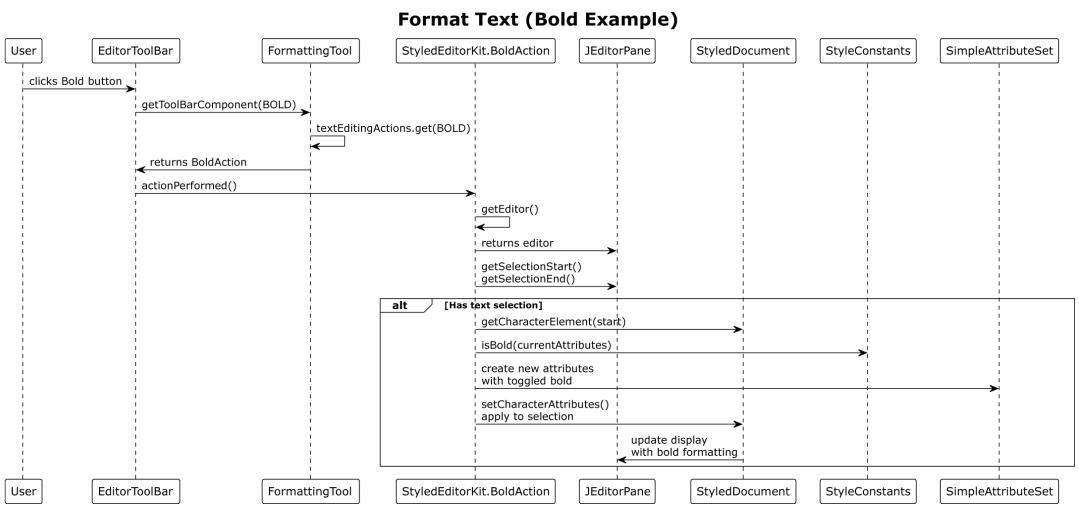

Figure 9: This sequence diagrams shows how text formatting attributes are applied to document selections, using bold formatting as an example (Link)

4.3 Find and Highlight Text

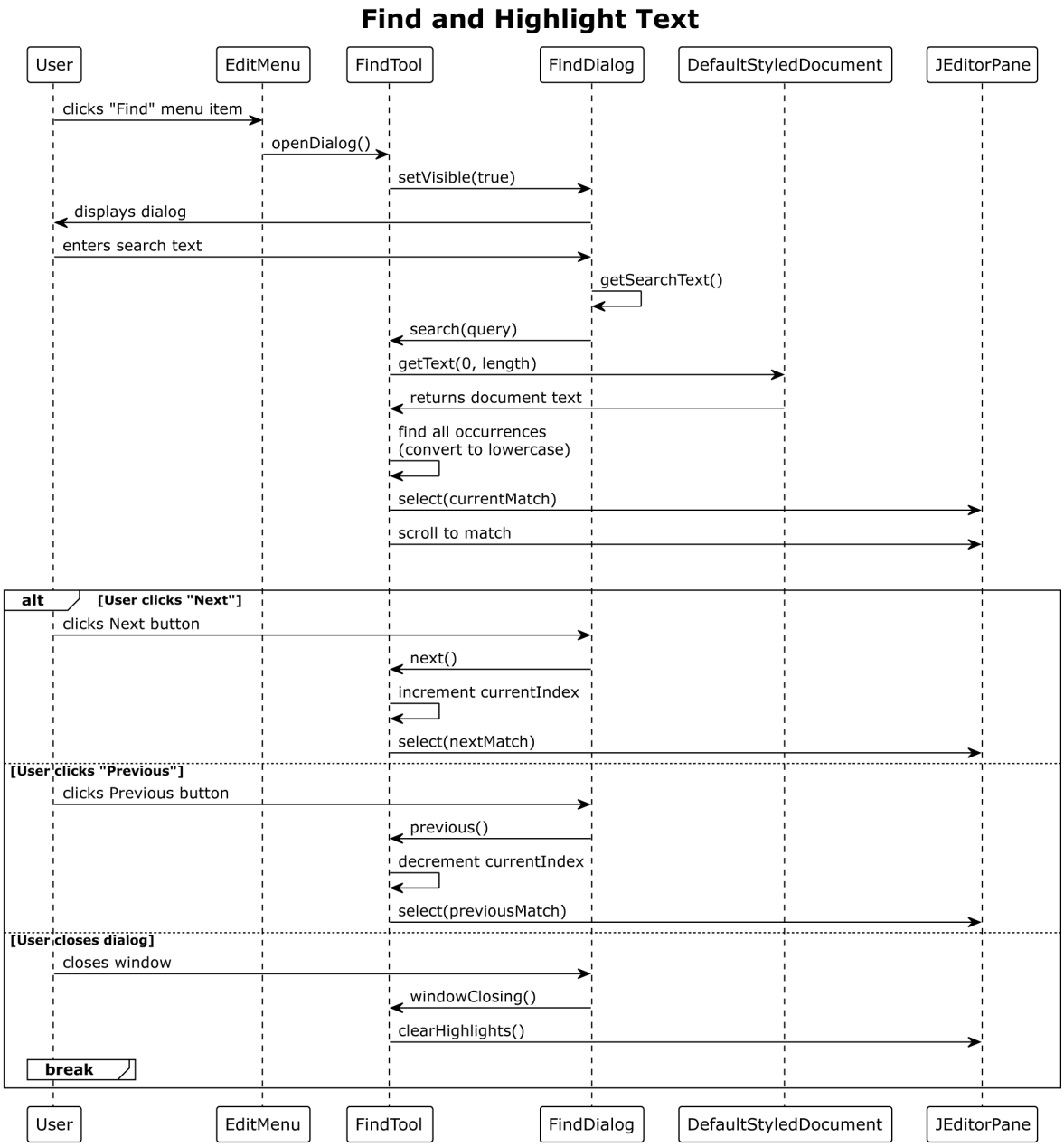

Figure 10: This diagram demonstrates the text search functionality workflow, from opening the find dialog to navigating between matches and managing result highlighting within the document (Link)

4.4 Edit Text

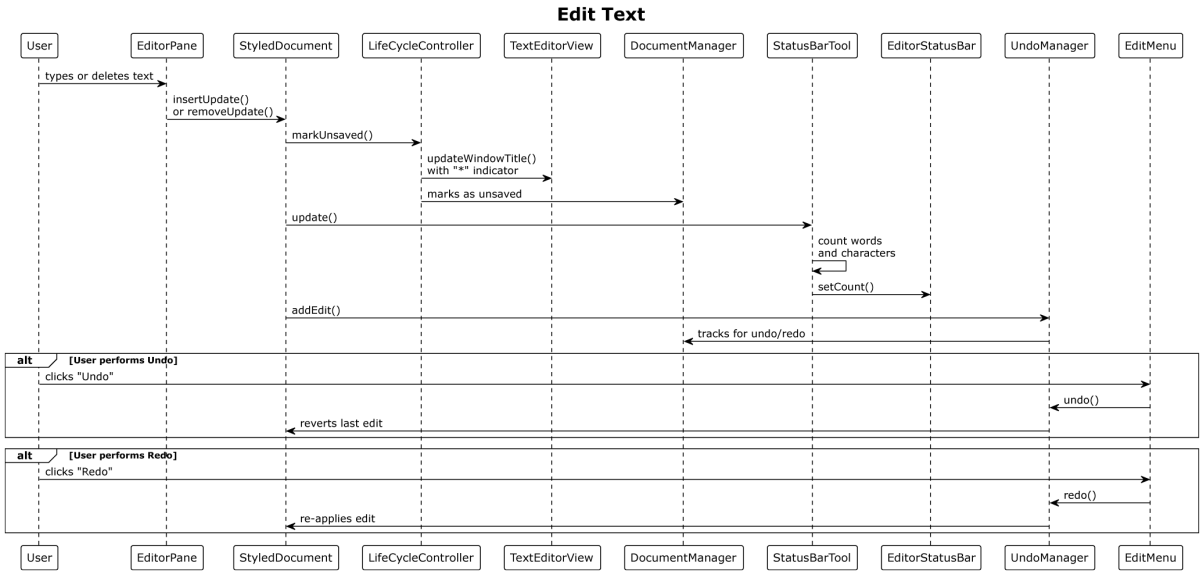

Figure 11: The diagram captures the text editing workflow, including content modification, change tracking, status updates, and undo/redo functionality management (Link)

4.5 Create/Open/Save Document

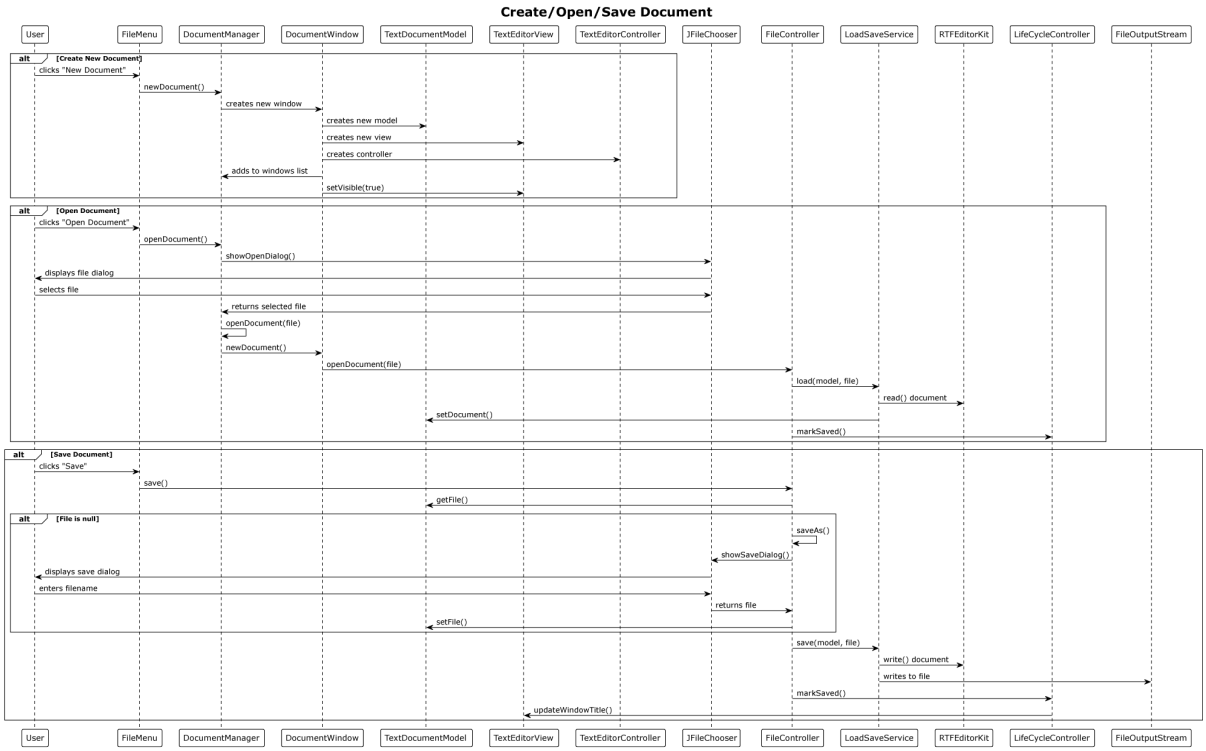

Figure 12: This sequence diagram presents the three primary document management operations, showing how new documents are created, existing files are opened, and current document are saved with proper state management (Link)

5 Unit Tests

This section presents the unit tests implemented in the program. It outlines the overall testing strategy, the goals of the tests, and the different cases covered for each component.

5.1 Testing Strategy

The project uses **JUnit 5** (JUnit Jupiter 5.13.4) to validate the components of the text editor. The goal of the testing is to ensure that each module behaves correctly. The tests emphasize three main goals:

- **Component-focused validation**: each unit test targets a single tool or subsystem (e.g. FindTool, Formatting actions, LoadSaveService), checking only its internal logic and expected outputs.
- **Normal and Edge Cases**: for every feature, both typical inputs and boundary conditions are tested, such as overlapping matches, empty documents, cleared formatting, or removing windows.
- **Minimal GUI**: GUI elements are created only when required by the component's API. The graphical interface itself is not tested, only the underlying logic.

5.2 DocumentManagerTest

The **DocumentManagerTest** verifies that the document management component correctly creation and removal of document windows within the application.

1. **Creating New Document**: checks that calling `newDocument()` returns a valid `DocumentWindow`, that the window contains a properly initialized model, and the list of open windows is updated correctly
2. **Closing a Document Removes It**: verifies that closing a window through closeWindow() successfully removes it from the manager's active windows list, while other open windows remain unchanged.

5.3 FindToolTest

The purpose of the **FindToolTest** suite is to verify that the **Find Tool** correctly locates all occurrences of a given string inside the model. These tests ensure that the tool behaves correctly across both normal and edge-case scenarios, and the result contain start-end index pairs for every match. The tests cover the following cases:

1. **Find Two Occurrences**: verifies that the tool correctly identifies multiple distinct matches in a short text. The Find Tool searches for `"A"` in the document which contains `"A B A"`. The test checks whether exactly two matches are found and if each match covers the correct range of the text.
2. **Find Nothing**: the document is empty, and the tool searches for `"A"` which naturally returns an empty result list.

3. **Find Same Letters**: focuses on repeated or overlapping matches. The document contains `"AAA"`, and the tool search for `"A"`. The expected behavior is that three matches should be found and they correspond to indexes `[0,1)`, `[1,2)`, and `[2,3)`.

4. **Text Case Insensitive**: the document contains `"Hello hElLo"`, and the tool searches for `"HEL"` using case-insensitive matching. Similarly to the previous test, the test checks if the two matches are found, and their respective indexes.

5. **No Match**: the document contains `"ABCDEF"` and the search term is `"Z"`, the result list is empty.

5.4 FormattingActionTest

The **FormattingActionTest** verifies that all custom formatting actions correctly apply visual and stylistic changes to the selected text inside the editor. Each test focuses on specific formatting feature and checks whether the expected attribute has been applied to the selection. The test is initialized using a `StyledEditorKit` and a `DefaultStyledDocument`, into which the string `"Testing"` is inserted. A dummy `JEditorPane` is created with this editor kit and document, and the text is selected.

1. **Change Font Action**: ensures that selecting a new font family updates the selected text with the expected font attribute (e.g. Arial).

2. **Clear Formatting Action**: checks that clearing formatting will reset all text attributes to the default values specified in `TextDefaults`.

3. **Font Size Action**: verifies that choosing a new font size correctly updated the size attribute of the selected text.

4. **Text Color Action**: confirms that applying a text color sets the correct foreground (text) color.

5. **Highlight Color Action**: ensures that the highlight/background color is correctly applied to the selected text

6. **Strikethrough Action**: checks whether action enables the strikethrough attribute.

7. **Subscript Action**: similarly, it verifies that the subscript attribute is applied when the action is performed.

8. **Superscript Action**: ensures that the superscript is applied correctly to the selected text

5.5 LoadSaveServiceTest

This test suite verifies that the documents are correctly saved to disk and loaded back into the editor using the **RTF-based** serialization. The test cover plain text, formatted text, and a edge case related to empty documents.

1. **Save and Load Plain Text**: checks that an unformatted document is saved and reloaded correctly, and no changes occurred as a consequence of serialization.

2. **Save and Load Formatted Text**: verifies that the formatting attributes (e.g. bold, italic, text color) are preserved when saving and loading a styled document. It should be noted that this format does not support many features and it is only expected to save the attributes that are supported by the format.

3. **Save and Load Empty Document**: ensures that saving and loading an empty documents behaves correctly.

5.6 StatusBarToolTest

The **StatusBarToolTest** suite verifies that the **Status Bar** component correctly reflects and tracks the current document state by updating the character and word counts displayed to the user. Each test creates a `TextDocumentModel`, creates a `TextEditorView`, assigns the model's document to the editor pane, and constructs a `StatusBarTool` instance. The tool's `update()` method is called to synchronize the status bar with the document content.

1. **Check Character Count**: ensures that `"Hello World"` results in a character count of 11 in the status bar.

2. **Check Word Count**: verifies that the string `"Hello World!  How are you?"` correctly distinguishes five words, and the status bar displays the value after updating.

3. **Check Empty Document**: confirms that an empty document produces both a word count and character count of zero.

References

[1] Stanislav Lapitsky, "Scaling with custom GlyphPainter in the JEditorPane/JTextPane." https://web.archive.org/web/20210512224543/http://java-sl.com/Scale_In_JEditorPane_GlyphPainter.html, 2008.